

TreeCoder: Systematic Exploration and Optimisation of Decoding and Constraints for LLM Code Generation

HENRIJS PRINCIS, University of Bristol, United Kingdom

ARINDAM SHARMA, University of Bristol, United Kingdom

CRISTINA DAVID, University of Bristol, United Kingdom

Large language models (LLMs) have shown remarkable ability to generate code, yet their outputs often violate syntactic or semantic constraints when guided only through natural language prompts. We introduce TREECODER, the most general and flexible framework to date for exploring decoding strategies, constraints, and hyperparameters in LLMs, and use it in code generation to enforce correctness and structure *during decoding* rather than relying on prompt engineering. TREECODER represents decoding as a tree search over candidate programs, where both decoding strategies and constraint functions—such as style, syntax, execution—are treated as first-class, optimisable components. This design enables systematic exploration and automatic tuning of decoding configurations using standard optimisation techniques. Experiments on Python, SQL and Rust show that TREECODER consistently improves accuracy across open-source models such as CodeLlama, Mistral, DeepSeek and Qwen, often significantly outperforming their unconstrained baselines.

CCS Concepts: • **Software and its engineering** → **Constraints**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Constrained Decoding, Code Generation, Large Language Models

ACM Reference Format:

Henrijs Princis, Arindam Sharma, and Cristina David. 2026. TreeCoder: Systematic Exploration and Optimisation of Decoding and Constraints for LLM Code Generation. *Proc. ACM Program. Lang.* 10, PLDI, Article 269 (June 2026), 30 pages. <https://doi.org/10.1145/3808347>

1 Introduction

Large language models (LLMs) have become the de facto standard for a wide range of code-related tasks, including code completion, summarisation and translation [4, 5, 12]. However, their outputs are often inconsistent, violating hard constraints such as the syntax of the target programming language [44] or introducing logical errors that break compilation or execution.

A straightforward way to mitigate these issues is to improve the base model, scaling up its parameters, extending training, or curating higher-quality data. However, these approaches are computationally expensive, slow to iterate and do not provide any guarantees. A more efficient alternative is to enhance model behaviour at inference time, through methods that adjust or constrain the decoding process. Such techniques fall broadly into two categories: (1) *Decoding strategies*, which adjust how the model explores its probability distribution during generation; and (2) *Constrained decoding*, which restricts outputs to those that satisfy external rules such as grammatical, syntactic or logical constraints.

Authors' Contact Information: [Henrijs Princis](#), University of Bristol, United Kingdom, de25328@bristol.ac.uk; [Arindam Sharma](#), University of Bristol, United Kingdom, arindam.sharma@bristol.ac.uk; [Cristina David](#), University of Bristol, United Kingdom, cristina.david@bristol.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/6-ART269

<https://doi.org/10.1145/3808347>

Decoding strategies. LLMs generate code token by token, a process known as *decoding*, by defining a conditional probability distribution $P_\theta(x_t \mid x_{<t})$ over the next token x_t given the preceding sequence $x_{<t}$.

A wide range of *decoding strategies* have been proposed to guide how language models (LMs) explore their predictive distribution during generation, each offering a different trade-off between efficiency, diversity, and correctness. The simplest approach, *sampling*, draws tokens stochastically from the model's conditional distribution to produce diverse outputs [11, 26, 50]. More structured alternatives such as *beam search* maintain the top- k most likely continuations at each step [21–23, 31, 32, 44]. Closely related greedy methods such as *best-first search* also prioritise high-likelihood continuations, but revisit earlier prefixes through backtracking when later validation rejects a selected branch. Recent work has adapted more exploratory planning techniques, notably *Monte Carlo Tree Search* (MCTS) and *Sequential Monte Carlo* (SMC), from games and Bayesian inference, showing promising results on mathematical reasoning and code synthesis [28, 40]. Each strategy embodies a distinct bias-variance compromise: sampling promotes diversity but may produce invalid code, greedy MAP-oriented methods such as beam search and best-first search prioritise likelihood at the cost of diversity, and population-based methods such as MCTS and SMC aim to balance exploration and exploitation under a fixed computational budget. No single method is uniformly optimal across all domains or decoding budgets.

While these decoding strategies determine *how* the model explores its distribution, they do not restrict *what* the model is allowed to produce. As a result, even high-likelihood sequences may violate the syntactic or semantic rules of the target language.

Constrained decoding. Contrary to this, *constrained decoding* [17, 28, 34, 36, 43, 44] modifies the decoding process by introducing a predicate $\Phi : \Sigma^* \rightarrow \{0, 1\}$ that specifies whether a partial sequence is admissible according to syntactic, semantic, or domain-specific rules. Formally, the constrained next-token distribution is $P_\theta^\Phi(x_t \mid x_{<t}) \propto P_\theta(x_t \mid x_{<t}) \cdot \Phi(x_{1:t})$. Only continuations satisfying $\Phi(x_{1:t}) = 1$ are considered during generation. As such, while decoding strategies control the search process itself, constraint-based approaches act orthogonally, enforcing external rules that restrict admissible outputs. For instance, *Adaptive Sampling with Approximate Expected Futures* (ASAp) used in Park et al. [34], is a *sampling* strategy that samples from a LM while ensuring outputs satisfy a *grammar constraint* and faithfully model the LM's conditional distribution over syntactically valid strings.

These methods are particularly valuable in *code generation*, where correctness is discrete and non-negotiable: any syntactic or logical violation immediately results in compilation or runtime failure.

Main challenges for constrained decoding. Despite rapid progress in decoding algorithms and constraint mechanisms, research on code generation remains fragmented. Each decoding strategy—sampling, best-first search, beam search, SMC, MCTS, ASAp—implements its own control loop, state representation and bookkeeping. Similarly, each constraint type tends to be integrated in an ad hoc manner, often coupled tightly to a specific decoding routine.

As decoding methods become increasingly specialised, researchers face two main challenges. First, *existing systems* [1, 16, 17, 29, 35] *hard-code particular combinations of search algorithms and constraint mechanisms*, making it difficult to isolate their individual contributions, compare their effects, or reuse and extend implementations. Second, *there is no principled way to systematically explore or optimise this growing space of alternatives*. Decoding strategies, constraints, and their hyperparameters interact in complex, non-linear ways, and small changes in

the way they are chosen can dramatically alter performance. As a result, many observed improvements are discovered through trial and error rather than through a structured, data-driven search over the design space.

The most well-known framework for training and inference with neural networks is Hugging Face [47]. While dominant for model training and inference, Hugging Face only supports beam search and sampling out of the decoding strategies we study, as it does not track a decoding tree, limiting support for backtracking methods such as best-first search and for broader tree-based decoding methods. IterGen [45] addresses this partially by introducing a decoding tree and grammar-symbol navigation. However, it decodes a single hypothesis at a time and tightly couples constraints with decoding, making it difficult to flexibly combine or systematically compare alternative algorithms and constraint configurations.

This work and contributions. We introduce TREECODER, a unified and extensible framework for exploring decoding strategies, constraints and hyperparameters in LLM-based code generation.

- ***TreeCoder’s main theoretical contribution is a unifying abstraction for decoding: trees of partial programs and expand-update-prune-move operators acting on those trees.*** We show that our abstraction generalises many state-of-the-art decoding strategies, both population- and trajectory-based. It transforms algorithm-specific implementations into composable variations of a common structure, enabling systematic comparison and optimisation.
- ***TreeCoder’s main technical contribution is the design and implementation of a modular framework built on this unified abstraction.*** It decouples decoding strategies, constraints, termination conditions, aggregation functions and optimisation into explicitly composable and comparable components. On the MBPP dataset [6], our implementation has little to no measurable overhead relative to bespoke HuggingFace implementations. Unlike prior systems like IterGen [45], TREECODER supports parallel hypotheses tracking and constraint-aware decoding in a single, extensible design.
- ***Empirically, one advantage of TreeCoder is that it can improve the performance of (smaller) open-weights models,*** which are particularly useful in scenarios where privacy or cost constraints favour local deployment.

Use of TREECODER in code generation. Most current interactions with LLMs for coding rely on “vibe coding”, steering the model through carefully worded prompts rather than explicit control. Developers embed requirements such as “only output valid Python code” or “use this API” in natural language, but these instructions are interpreted heuristically and often ignored, especially when they conflict with the model’s learned biases or extend across multiple generation steps.

For users seeking more control, TREECODER replaces this trial-and-error process with a principled alternative: applying constraints *during decoding* rather than merely describing them in the prompt. In this paradigm, stylistic, syntactic and semantic rules become first-class, programmable components that dynamically steer the model’s generation process, an approach particularly well-suited to code generation, which must satisfy strict syntactic and semantic correctness. TREECODER offers a comprehensive library of such constraints and users can define new ones, making it easy to prototype, combine and experiment with new decoding behaviours.

Beyond constraints, TREECODER also allows users to interchange decoding strategies and, when datasets are available, to automatically optimise configurations of decoding strategies, constraints and hyperparameters, finding the best setting for a given coding task.

Experimental results. We evaluated the impact of TREECODER on code generation across small open models (CODELLAMA-7B & CODELLAMA-13B [42], MISTRAL-7B-INSTRUCT-v0.3 [25], DEEPSEEK-CODER-7B-INSTRUCT-v1.5 [14], DEEPSEEK-R1-7B [20] and Qwen2.5-Coder-1.5B [48]). We considered

generation tasks for Python (MBPP [6] and LiveCodeBench [24]), SQL (SQLSpider [49]) and Rust (translated MBPP [33]). Across all models and tasks, TREECODER yields consistent improvements in accuracy. Moreover, through automatic optimisation of decoding strategies, constraint sets and hyperparameters, TREECODER discovers task-specific configurations that further enhance performance.

2 Background

In this section, we introduce some background concepts and terminology used throughout the paper. For this purpose, Table 1 presents a taxonomy of decoding functions, classifying representative algorithms along five orthogonal dimensions that characterise most existing decoding strategies.

(i) **Determinism**-whether the search procedure is stochastic (e.g., sampling) or deterministic (e.g., beam search or best-first search). (ii) **Concurrency**-whether multiple hypotheses are expanded in parallel (*population-based*) or a single trajectory is explored at a time (*trajectory-based*). (iii) **Backtracking**-whether the algorithm can revisit earlier states to explore unvisited branches. (iv) **Roll-outs**-whether candidate continuations are recursively expanded to full completions before being evaluated; in other words, a *roll-out* simulates entire trajectories from a partial hypothesis to assess its long-term promise. (v) **Optimisation goal**-whether the algorithm seeks a single most probable completion (*MAP-oriented*) or approximates the underlying model distribution (*distribution-oriented*).

MAP-oriented methods-such as beam search, best-first search, and MCTS-prioritise the most likely program under model and constraint scores. *Distribution-approximating methods*-such as sampling, SMC, and ASAp, aim to capture diverse plausible continuations, approximating the constrained posterior rather than a single mode. Roll-out-based algorithms like MCTS and ASAp both backtrack and expand candidates to completion, while methods like best-first search perform backtracking without full roll-outs.

Decoding Function	Determinism	Concurrency	Backtracking	Roll-outs	Goal Type
Beam Search	Deterministic	Population	No	No	MAP
SMC	Stochastic	Population	No	No	Distribution
Sampling	Stochastic	Population	No	No	Distribution
MCTS	Stochastic	Trajectory	Yes	Yes	MAP
ASAp	Stochastic	Trajectory	Yes	Yes	Distribution
Best-First Search	Deterministic	Trajectory	Yes	No	MAP

Table 1. Categorisation of decoding functions.

Instantiation within our framework. To demonstrate the generality of our framework, we instantiate six representative decoding techniques: beam search, SMC, sampling, MCTS, ASAp and best-first search. Each is implemented using shared modular components for search policy and constraint enforcement, showing that diverse decoding algorithms and constraint combinations can be expressed within a single tree search abstraction.

3 Framework for Constrained Decoding

At a high level, the framework treats constrained code generation as a form of tree search that is guided jointly by the LLM and user-specified constraints (Figure 1). Structural components, such as the decoding tree, its nodes, and their internal states, define the shared substrate over which all algorithms operate. Pluggable components such as the decoding function, termination condition, aggregation function, constraint modules and optimisation algorithm encapsulate the policy, stopping criteria, and external rules that govern traversal of the tree. By separating these structural and pluggable elements, the framework allows users to compose and compare decoding

strategies systematically, to optimise configurations automatically (e.g., via Bayesian optimisation), and to extend existing implementations with minimal code.

We next provide a high-level description of the framework’s components. We begin with the structural elements that underpin all decoding algorithms, followed by the pluggable components that control and constrain the generation process.

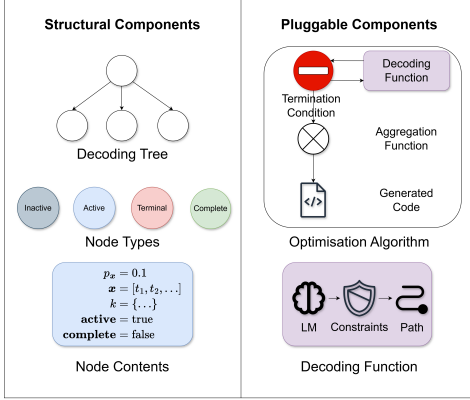


Fig. 1. High-level illustration of our framework. Structural components including the decoding tree, nodes, and node contents are on the left. Pluggable components including decoding function, termination condition, aggregation function and optimisation algorithm are on the right. The decoding function consists of a language model and an optional set of constraints.

Structural Components.

- A decoding **tree** is the *unifying abstraction* across our algorithms that minimises code duplication. Every algorithm makes use of the root node, which determines where the LM’s generation starts as well as methods for accessing active, inactive, terminal and complete nodes.
- To enable *systematic exploration of the solution space*, we require that every **node** contains links to parent and child nodes and that they contain a score s that indicates their quality. The score is usually equal to the node’s probability p_x , but more generally it is calculated from the state of the tree. For example, for MCTS the score is based on number of times the node has been visited and led to a solution that did not violate constraints.
- To enable *grammar symbol navigation* (i.e. the ability to track and reason about the current position within a program’s grammar as tokens are generated), upon every application of a constraint function such as an incremental parser, our framework automatically stores the constraint function’s state within our node’s state k . This state can then be used by the decoding function to make decisions such as how far to backtrack or to decide which branch of the search tree to investigate next.

We categorise each node as being either terminal, complete, active or inactive. We expand all active nodes concurrently which enables our framework to *simultaneously explore multiple hypotheses*. Furthermore, this categorisation serves as an interface between the decoding function and the termination condition.

- **Terminal nodes** have a score of zero and cannot be expanded. They represent a point where a constraint violation has been detected.
- **Complete nodes** contain the end-of-sequence token and have a score greater than zero. They also cannot be expanded.
- **Active nodes** are non-terminal nodes that will be expanded in the next iteration of the decoding function.
- **Inactive nodes** do not contain an end-of-sequence token and usually are not expanded by the subsequent application of the decoding function.

Pluggable Components. In our framework, the constraint functions, decoding functions, termination conditions, aggregation functions, LM and the optimisation algorithm denote modular components that can be plugged in. We provide a library of implementations for the first four, and users are free to experiment with custom versions as well as with different LMs and optimisation algorithms.

- An **optimisation algorithm** that searches over the rest of the pluggable components and optimises a user-provided objective function.
- A possibly empty set of **constraint functions** Φ that guide the LM towards generating good quality code either by reweighting candidate continuations or by validating selected continuations lazily.
- A **decoding function** δ that iteratively grows the tree by expanding the active nodes using the LM and, depending on the algorithm, either eagerly or lazily applying the constraints until a terminal condition is reached.
- A **termination condition** ρ that checks whether the tree is complete.
- An **aggregation function** $\otimes$ that aggregates nodes within the tree to produce the final output.
- An auto-regressive **LM** that given a sequence of tokens defines the probability distribution over a single token continuation.

The pseudocode for our approach is given in Algorithm 1 and defines the core loop of our framework, which treats decoding as a tree search guided by a LM and constraint functions. Starting from a root node, the framework repeatedly expands active nodes using a decoding function, δ , until a termination condition, $\rho(Tree)$, is met. Depending on the decoding strategy, constraints can either be incorporated eagerly into candidate scoring or evaluated lazily only on the continuation selected for expansion, as in best-first search. Once the tree is complete, an aggregation function, $\otimes(Tree)$, selects the best set of candidates.

Algorithm 1: TREECODER

Require: δ : Decoding function
Require: Φ : Optional set of constraint functions
Require: ρ : Terminal condition
Require: $\otimes$: Aggregation function
Require: LM: A trained language model

```

1:  $n \leftarrow \text{Node}(x = [], p_x = 1, \text{score} = 1)$    $\triangleright$  The starting node contains an empty sequence of tokens, probability of 1, and a score of 1
2:  $Tree \leftarrow \text{create\_tree}(n)$    $\triangleright$  Tree is created with root node
3: while  $\rho(Tree) \neq \text{true}$  do   $\triangleright$  While tree is incomplete
4:    $Tree \leftarrow \delta(Tree, LM, \Phi)$    $\triangleright$  Extend it by using the decoding function
5: end while
6: return  $\otimes(Tree)$    $\triangleright$  Once tree is complete, return the most promising sequences by aggregating the tree
```

In the rest of the section, we provide details about the individual components of the framework.

3.1 Optimisation Algorithm

TREECODER includes a higher-level optimisation module that treats decoding functions, constraint sets and hyperparameters as elements of a joint search space and applies standard optimisation methods such as grid search, random search, or Bayesian optimisation to identify effective configurations. By abstracting optimisation as an independent module, TREECODER cleanly separates search logic from decoding, enabling systematic and reusable tuning across tasks.

Algorithm 2 outlines our implementation. At each iteration, the optimiser proposes a new parameter configuration based on past evaluations, executes TREECODER with these parameters, and records the resulting performance. After a predefined number of iterations, the best-performing configuration is returned. This design cleanly separates optimisation logic from decoding and provides a minimal interface, enabling seamless integration with libraries such as Optuna [2].

Algorithm 2: Optimisation Algorithm

Require: θ : The ranges of hyper-parameters to optimise (e.g., decoding functions or population size range)
Require: `optimise`: A function that proposes new parameter instantiations based on past evaluations
Require: `max_iters`: The maximum number of optimisation iterations

```

1:  $past\_accuracies \leftarrow \emptyset$ 
2:  $past\_params \leftarrow \emptyset$ 
3: while  $i < max\_iters$  do
4:    $params \leftarrow \text{optimise}(\theta, past\_accuracies, past\_params)$ 
5:    $accuracy \leftarrow \text{TreeCoder}(params)$ 
6:    $past\_accuracies \cup \{accuracy\}$ 
7:    $past\_params \cup \{params\}$ 
8:    $i++$ 
9: end while
10: return  $\arg \max_{p \in past\_params} \text{value}(p)$   $\triangleright$  Return the best instantiation of the parameters the optimisation algorithm was able to find

```

3.2 Decoding Function δ

The decoding function is a pluggable component that iteratively expands the decoding tree until a termination condition is met, as outlined in Algorithm 1. Its implementation follows four core stages: *expand*, which grows active nodes using the LM’s probability distribution or performs roll-outs; *update*, which revises node scores based on the optimisation goal (MAP or distribution approximation); *prune*, which removes low-scoring branches and manages memory by clearing cached states; and *move*, which selects the next active nodes, determining whether the algorithm performs backtracking and whether it operates in a population- or trajectory-based manner. This unified structure allows diverse algorithms to share a common control loop (Algorithm 3) while varying in their specific implementations of these stages. We next explain each stage in more detail.

Algorithm 3: δ – General decoding function

Require: *Tree*: The current search tree
Require: *LM*: Language model
Require: Φ : Possibly empty set of constraints

```

1:  $children \leftarrow \text{Expand}(N, LM, \Phi)$  or  $\text{Rollout}(N, LM, \Phi, \delta)$   $\triangleright$  Expand step
2:  $children \leftarrow \text{LocalUpdate}(children)$   $\triangleright$  Update children’s scores
3:  $Tree \leftarrow \text{GlobalUpdate}(Tree)$   $\triangleright$  Update non-child node states
4:  $Tree \leftarrow \text{Prune}(Tree)$   $\triangleright$  Prune non-active nodes
5:  $Tree \leftarrow \text{Move}(Tree)$   $\triangleright$  Select new set of active nodes
6: return Tree

```

Expand. The expand step (line 1 in Algorithm 3) either expands the set of active nodes or performs a *roll-out* which is a recursive application of the general decoding function.

Let N be a set of nodes to be expanded and $n \in N$ denote a single node. The Expand operator applies the expansion procedure to each node in N and returns the union of their individual expansions (see Equation 1).

We denote the set of n ’s children as $\text{Expand}(n, LM, \Phi)$. Each child node is defined by a token sequence $\mathbf{x} = n_{\mathbf{x}} \circ t$, where $\circ$ denotes concatenation and $t \in \mathcal{V}$ is a token from the LM’s vocabulary (see Equation 2).

The child’s probability is computed as $p_{\mathbf{x}} = p_{\text{parent}} \cdot p(t \mid n_{\mathbf{x}}) \cdot \Phi(n_{\mathbf{x}} \circ t)$ divided by a normalising constant that ensures the probabilities of all children sum to p_{parent} . In the case that no continuation is valid according to the set of constraint functions, we set all child probabilities to zero without normalizing to avoid division by zero. Here p_{parent} is the probability of the parent node that’s being expanded, $p(t \mid n_{\mathbf{x}})$ is the conditional probability of choosing token t and Φ is the aggregate score given by a set of constraint functions (see Equation 3).

Finally, to calculate the product of experts score given by the set of constraints Φ , we take their product (see Equation 4).

$$\text{Expand}(N, \text{LM}, \Phi) = \bigcup_{n \in N} \text{Expand}(n, \text{LM}, \Phi) \quad (1) \quad \text{Expand}(n, \text{LM}, \Phi) = \{(x = n_x \circ t, p_x) \mid t \in \mathcal{V}\} \quad (2)$$

$$p_x = p_{\text{parent}} \cdot \frac{p(t \mid n_x) \cdot \Phi(n_x \circ t)}{\sum_{t' \in \mathcal{V}} p(t' \mid n_x) \cdot \Phi(n_x \circ t')} \quad (3) \quad \Phi(x \circ t) = \prod_{i=1}^n \phi_i(x \circ t) \quad (4)$$

Approximate Expand. While equations 1 to 4 provide a conceptually accurate description of node expansion, in practice modern LM vocabularies are so large (32k tokens for Llama 3.1) that it is impractical to evaluate a constraint function over all possible one token continuations. Furthermore, creating a large number of nodes on each expansion would quickly run into memory limitations.

To alleviate these issues, we adapt a procedure inspired by SMC [28] where a representative subset of particles approximates the true constrained distribution. In particular, we use the approximate expansion procedure below to approximate the true-weighted distribution:

- (1) Generate the LM's distribution of single token continuations of the current node.
- (2) Draw j samples from this distribution.
- (3) Evaluate the constraint function on these samples.

The generated subset in most practical circumstances provides a good approximation to the set of all nodes given by Equation 2. This is because when j is the size of the LM's vocabulary $|\mathcal{V}|$ and nodes are sampled without replacement, the approximate expansion procedure is equivalent to Equation 2.

This approximation can cause incompleteness when constraint functions are poorly aligned with the LM's token probabilities—for example, when the prompt asks for Python code but decoding is constrained to Java. It may also be inaccurate when $j \ll |\mathcal{V}|$, since a large portion of the true probability mass may lie outside the sampled tokens.

To improve the approximation when j is small, we support two sampling modes in step 2: *greedy*, which selects the j most likely tokens according to the LM (top- k), and *stochastic*, which samples j tokens from the LM's original distribution. Greedy sampling suits MAP-oriented decoding methods such as best-first search, beam search and MCTS, while stochastic sampling is better aligned with distribution-based methods such as constrained sampling or SMC.

We also provide an optional token allow-list that guarantees certain tokens are always considered during expansion. This is particularly useful in highly constrained domains where the LM is poorly aligned with the target language (see Section 7.1). The parameter j therefore exposes an explicit trade-off between efficiency and completeness.

Finally, different tokenisations can produce the same string (e.g. "ab" vs. "a"+"b"), increasing the chance of reaching correct programs through alternative paths.

Roll-outs. For algorithms without roll-outs, all active nodes are expanded once and the step is complete. For algorithms with roll-outs, active nodes are iteratively expanded until an end condition is met (e.g., generation of the EOS token). Conceptually, a roll-out can be viewed as a recursive invocation of the framework, where the inner call is restricted to a non-roll-out decoding function to prevent unbounded recursion. In our implementation, this recursion is unrolled into a loop for clarity.

Update. The update step updates node's scores and states as shown in lines 2-3 in Algorithm 3. The score calculation determines the algorithm's *goal*. Usually algorithms score nodes according to their probability. Sampling, beam search and best-first all fall into this category. MCTS, ASAp and SMC all use more elaborate scoring schemes. For example, MCTS uses the PUCT score [9] which

calculates the node's score based on the probability, number of times a roll-out led to a completion that satisfies constraints, and the number of times the node and its parent has been visited.

There are two types of updates *local updates* and *global updates*. Local updates update the state of newly extended nodes. Global updates update the state of all other nodes in the tree. For example, for MCTS the global update step updates the entire lineage of rolled-out nodes.

Prune. Its purpose is memory management. For example, for beamsearch of width k , after the *Extend loop* we will have k^2 possible continuations. However, only the k highest scoring continuations are used, therefore we can prune the rest. Similarly SMC, checks if the scores of the nodes fall below a certain threshold and then conditionally resamples them. Particles that were not chosen after resampling can be pruned.

Move. The move step of the algorithm selects which nodes will be expanded next. Population based algorithms always move to the newly extended nodes. However, trajectory based algorithms can backtrack to earlier state. For example, the ASAp algorithm never moves away from the start node. Best-first search greedily follows the highest scoring branch of the tree, but when a selected token fails validation it backtracks to the beginning of the last line and continues from a different frontier node whose score has not been downweighted.

3.3 Constraint Function Φ

While any function that imposes constraints on code generation could be implemented directly as a decoding function, we find it useful to draw a conceptual distinction between the decoding function that does the high-level decision making for which branch to explore next and the constraint function guiding the LM towards more promising solutions. This distinction makes our framework *modular* and allows swapping out constraint functions and decoding functions. It also allows different decoding functions to invoke the same constraints differently, for example by applying them to every candidate continuation or lazily only to the continuation that was selected for expansion.

To handle both soft constraints (i.e. preferences that guide generation without strictly invalidating outputs) and hard constraints (i.e. rules that must always be satisfied), we follow the idea of potential functions in [28], such that our constraint function assigns a quality score to partial or complete token sequences. This design enables a unified mechanism for constraint enforcement: the update and prune stages of the decoding function can replicate hard masking behaviour (assigning zero score to invalid continuations) or soft masking (down-weighting unlikely continuations, as in [28]).

We further adopt the product-of-experts formulation which means that Φ can be decomposed into individual constraints ϕ such that $\Phi = \prod_{i=1}^n \phi_i$ where each ϕ_i takes as input the sequence of tokens $\mathbf{x}$ and outputs a score indicating the quality of the sequence of tokens. For example, a score of 0 would indicate that the solution is invalid.

Product of experts formulation is desirable because it encourages breaking down the score assessment into individual modular components that can be applied individually or stacked. Such modularity helps allow for more explainable systems where the end-user can see which constraint has been violated. By using multiplication as the aggregation function any expert gets veto power over the sequence of tokens. This aligns well with the setting of constrained decoding for code generation where any syntax mistake, unit test failure, or security violation invalidates the solution.

Increased Expressive Power. Furthermore, we find that the product of experts formulation coupled with grammar constrained decoding increases expressivity of context free languages. It allows us to recognise strings that a single context-free grammar (CFG) could not. For example, consider the following two grammars over the alphabet $\{a, b, c\}$:

$$\begin{array}{ll}
G_1 : S_1 \rightarrow AB & G_2 : S_2 \rightarrow CD \\
A \rightarrow aA \mid \epsilon & C \rightarrow aCb \mid \epsilon \\
B \rightarrow bBc \mid \epsilon & D \rightarrow cD \mid \epsilon
\end{array}$$

Grammar G_1 and G_2 generate the language L_1 and L_2 respectively. L_1 consists of any number of a 's followed by an equal number of b 's and c 's. Similarly L_2 consists of an equal number of a 's and b 's followed by any number of c 's. Their intersection enforces both constraints simultaneously: $L_1 = \{a^i b^n c^n \mid i, n \geq 0\}$; $L_2 = \{a^n b^n c^j \mid n, j \geq 0\}$; $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$. This language, well known from formal language theory, is *not* context-free [8]. Hence, while both L_1 and L_2 are individually generated by CFGs, their intersection exceeds the expressive power of context-free grammars. Our approach, however, can represent this intersection naturally by adding two context-free grammars as decoding constraints to the `expand_node` function.

3.4 Termination Condition ρ

This function determines whether the tree is complete. If so, it returns all terminal nodes along with their associated scores. The condition is typically based on either the number of terminal nodes, length of the solution or a set time constraint. For example, for beam search of size k , the termination condition ρ is reached after k terminal nodes have been found, i.e. $\rho: (\#\{n \mid n \text{ is terminal}\}) \geq k$.

3.5 Aggregation Function $\otimes$

The aggregation function combines all terminal nodes to produce an output. Two common aggregation functions are max based aggregation which chooses the top- k highest scoring nodes and *count* based aggregation which chooses the most common node.

4 Examples of Decoding Functions

While enumerating all of them is impractical, our library implements three widely used algorithms—best-first search, beam search and sampling—and three emerging approaches—MCTS, SMC, and ASAP—demonstrating the generality of our framework.

To illustrate our explanation of a decoding function, we compare the implementation of different decoding functions within our framework using the four steps of *expand*, *update*, *prune*, and *move*.

Expand. In Figure 2 we see the six implementations of the *expand*. On the left side, we show decoding functions without roll-outs. Functions without roll-outs differ only in the way they use the *Expand* function. In particular, beam search and best-first search are MAP oriented methods and therefore use greedy approximation, whereas sampling and SMC are both distribution approximating methods and therefore use the stochastic approximation.

Each decoding function draws a different number of children. SMC always draws a single child node and creates duplicates in the update step if the sampled node violates constraints. Sampling adjusts for lower population size by directly drawing $\left\lceil \frac{k}{|N|} \right\rceil$ children, beam search always draws k children per node resulting in k^2 child nodes out of which only k will be kept, and best-first search also draws up to k greedy candidates.

On the right side there are two decoding functions that use rollouts. This means they go through all of the four stages of *expand*, *update*, *prune*, *move* within the *expand* stage.

For MCTS this is done in the while loop (lines 4 to 11). I.e. line 6 is *expand*, line 7-8 is *update*, line 9 performs *pruning*, and line 10 corresponds to *move*. The only difference between ASAP and MCTS is in how they score nodes within the *expand*.

Expand

Require: *Tree*, *LM*, Φ : Possibly empty set of constraints, *k*: Tree width constraint

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $\text{greedy} \leftarrow \text{True}$ 
3:  $j \leftarrow k$ 
4:  $M \leftarrow \text{Expand}(N, \text{LM}, \Phi, \text{greedy}, j)$ 

```

(a) Beam Search

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $\text{greedy} \leftarrow \text{False}$ 
3:  $j \leftarrow \left\lceil \frac{k}{|N|} \right\rceil$ 
4:  $M \leftarrow \text{Expand}(N, \text{LM}, \Phi, \text{greedy}, j)$ 

```

(b) Sampling

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $\text{greedy} \leftarrow \text{False}$ 
3:  $j \leftarrow 1$ 
4:  $M \leftarrow \text{Expand}(N, \text{LM}, \Phi, \text{greedy}, j)$ 

```

(c) SMC

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $\text{greedy} \leftarrow \text{True}$ 
3:  $M \leftarrow \text{Expand}(N, \text{LM}, \{\}, \text{greedy}, k)$ 

```

(d) Best-First Search

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $N \leftarrow \text{PUCT}(N)$ 
3:  $m \leftarrow \text{sample}(N, N.\text{scores})$ 
4: while not  $m.\text{is\_complete}$  do
5:    $m.\text{is\_active} \leftarrow \text{False}$ 
6:    $M \leftarrow \text{Expand}(N, \text{LM}, \Phi, \text{True}, k)$ 
7:    $M \leftarrow \text{PUCT}(M)$ 
8:    $m \leftarrow \text{sample}(M, M.\text{scores})$ 
9:    $\text{tree.prune\_kv\_cache}()$ 
10:   $m.\text{is\_active} \leftarrow \text{True}$ 
11: end while

```

(e) MCTS

```

1:  $N \leftarrow \text{get\_active\_nodes}(\text{Tree})$ 
2:  $m \leftarrow N[0]$ 
3: while not  $m.\text{is\_complete}$  do
4:    $m.\text{is\_active} \leftarrow \text{False}$ 
5:    $M \leftarrow \text{Expand}(m, \text{LM}, \Phi, \text{False}, k)$ 
6:    $M \leftarrow \text{EFG}(M)$ 
7:    $m \leftarrow \text{sample}(M, M.\text{scores})$ 
8:    $\text{tree.prune\_kv\_cache}()$ 
9:    $m.\text{is\_active} \leftarrow \text{True}$ 
10: end while

```

(f) ASAP

Fig. 2. Expand without rollouts (left) and expand with rollouts (right). The *sample* function returns a node from the distribution defined by normalizing the node's scores to sum to 1. The *PUCT* function calculates the *PUCT* score [41] and *EFG* calculates the node's expected future grammaticality [34].

Another important consideration when implementing a rollout based decoding function is to choose the right termination condition within the while loop. Simply relying on sampling the EOS token is not sufficient because if a mistake has been made at the beginning of a Python program, then sampling the EOS token would lead to a probability of zero being assigned by the Expand function. This could then cause the terminal node to be never selected for expansion resulting in an infinite loop. This is captured by the completeness check for MCTS (line 4) and ASAP (line 3).

Update. The *update* part of the algorithm performs both *local* and *global* updates. It is responsible for scoring nodes and updating node specific state. In Figure 3, we see that the update step behaves differently for each of the decoding functions. For sampling and beam search, it scores nodes using their probability (lines 1-3). These would be considered local updates since the score calculation is only based on the newly extended nodes.

Similarly SMC only applies local updates. Node score is either 0 or 1 (line 2) depending on whether it satisfies all constraints. If the effective sample size (*s*) falls below a threshold (0.6), then nodes are resampled according to their scores to the population size (lines 5-8). This step ensures the population size remains consistent after an application of the decoding function.

Best-first search performs a local update by scoring nodes according to their probability (line 2). It then conditionally applies a global update based on whether the top scoring continuation satisfies constraints Φ . If the constraints are not satisfied the algorithm backtracks to the beginning of the last line and multiplicatively downweights the scores along the rejected suffix so that the

Update

```

1: for  $m \in M$  do
2:    $m.score \leftarrow m.probability$ 
3: end for
4:  $M \leftarrow sorted(M, M.score)$ 

```

(a) Beam Search

```

1: for  $m \in M$  do
2:    $m.score \leftarrow m.probability$ 
3: end for

```

(b) Sampling

```

1: for  $m \in M$  do
2:    $m.score \leftarrow m.score \times [m.probability]$ 
3: end for
4:  $s \leftarrow effective\_sample\_size(Nodes)$ 
5: if  $s \leq 0.6$  then
6:    $M.scores \leftarrow \frac{\sum M.scores}{k}$ 
7:    $M \leftarrow reasample(M, M.scores, k)$ 
8: end if

```

(c) SMC

```

1:  $is\_win \leftarrow [m.probability]$ 
2: while  $m.parent$  do
3:    $m.playouts \leftarrow m.playouts + 1$ 
4:    $m.wins \leftarrow m.wins + is\_win$ 
5:    $m.score \leftarrow PUCT(m)$ 
6:    $m \leftarrow m.parent$ 
7: end while

```

(d) MCTS

```

1: while  $m.parent$  do
2:    $efg \leftarrow 0$ 
3:   for  $c \in m.children$  do
4:      $efg \leftarrow efg + c.probability \times c.efg$ 
5:   end for
6:    $m.efg \leftarrow efg$ 
7:    $m \leftarrow m.parent$ 
8: end while

```

(e) ASAP

```

1:  $m \leftarrow M[0]$   $\triangleright$  Select the highest-probability continuation
2:  $m.score \leftarrow m.probability$ 
3: if  $\Phi(m) = 0$  then
4:    $n \leftarrow backtrack\_to\_start\_line(m)$ 
5:    $downweight\_path(n, m)$ 
6:    $m.score \leftarrow 0$ 
7:    $m \leftarrow n$ 
8: end if

```

(f) Best-First Search

Fig. 3. Update step of all six decoding functions

same invalid trajectory is less likely to be selected again (lines 3-8). This lazy validation strategy is faster while remaining faithful for hard constraints.

MCTS and ASAP both employ *global updates*. They both use backpropagation (lines 2-7 and 1-8) respectively to update the entire lineage of node's scores and state. For MCTS node's state includes number of wins (i.e. number of times the node led to a valid completion). For ASAP, node's state includes expected future grammaticality (EFG). Note that upon node creation, we initialise playouts and wins to be equal to zero and EFG to be equal to 1.

Prune. In Figure 4 we see that the prune step's purpose is to manage memory. The memory footprint required for storing each node's state is relatively small when using our approximation. However, KV (Key-Value) cache [27] takes up a lot of space (up to 0.5GB in this work, exact size varies based on model size, architecture, and sequence length) and thus needed to be cleared on every application of decoding function to avoid out-of-memory errors. Therefore, after each update step, we perform a call to *tree.prune_kv_cache()* which clears the KV cache of all inactive nodes. To maintain fast decoding, it keeps the cache of active nodes that will be expanded within the next iteration.

Move. The move step of the algorithm selects the next node to expand. We see that population based methods (Beam Search, Sampling and SMC) all move. I.e. they change which nodes are active after an application of the decoding function. MCTS and ASAP are both stationary meaning that they never move from the tree's root node. Best-first search is trajectory-based: it activates the highest scoring expandable node in the tree, which may lie on a different branch after a rejected continuation has been downweighted and the search has backtracked to the start of the last line.

Prune & Move

```
1:  $M \leftarrow M[0:k]$ 
2:  $tree.prune\_kv\_cache()$ 
3: return  $set\_active(M)$ 
```

(a) Beam Search

```
1:  $M \leftarrow \{m | m \in M \cap m.score \geq 0\}$ 
2:  $tree.prune\_kv\_cache()$ 
3: return  $set\_active(M)$ 
```

(b) Sampling

```
1:  $tree.prune\_kv\_cache()$ 
2: return  $set\_active(M)$ 
```

(c) SMC

```
1:  $tree.prune\_kv\_cache()$ 
2: return  $set\_active(tree.root\_node)$ 
```

(d) MCTS

```
1:  $tree.prune\_kv\_cache()$ 
2: return  $set\_active(tree.root\_node)$ 
```

(e) ASAP

```
1:  $tree.prune\_kv\_cache()$ 
2: return  $set\_active(\{m\})$ 
```

(f) Best-First Search

Fig. 4. Prune and move steps of all six decoding functions

Even though MCTS and ASAP remain at the root during move, their rolled-out sequences are still stored within the tree and used to calculate the end condition ρ and within the aggregation function $\otimes$ to allow the framework to terminate and return the most promising sequences.

5 Examples of Constraint Functions

In this section, we provide examples of the types of constraint functions for our framework. For illustration, Figure 6 provides several constraint functions that can be combined within our framework. Each constraint function takes in a sequence of tokens generated so far, a tokenizer, prefix length and optional state that contains additional information about the problem such as unit tests. Then, it evaluates the partially generated sequence and returns a scalar score indicating its validity. For instance, SYNTAX checks whether the decoded fragment can be compiled using Python’s codeop module, NO-COMMENTS suppresses inclusion of comments, NO-EXPLANATIONS enforces that outputs start with def and contain no capital letters after the first line, nudging the model toward code-like continuations rather than natural language explanations, and UNIT-TESTS executes generated code against extracted assertions.

Beyond those in Figure 6, we define several additional constraint types that capture increasingly stricter properties of the generated programs (for brevity, these are not provided):

(i) CORRECT-SIGNATURE—checks whether the function names match the specification and have the correct number of arguments.

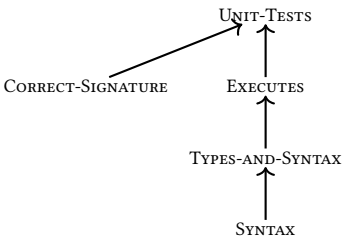


Fig. 5. Partial order of constraints.

(ii) TYPES-AND-SYNTAX—applies the Pyright static analyser [13] to detect type and syntax errors. For this purpose, we require the model to generate explicit type annotations. (iii) EXECUTES—verifies whether the function executes without errors when provided the correct arguments as input. Besides static errors (e.g. syntax, types), this has the ability to catch runtime errors such as infinite loops or unhandled exceptions.

These constraints form a hierarchy of restrictiveness, with UNIT-TESTS being the strongest (while NO-COMMENTS acts orthogonally):

Require: *token_ids*: Model-generated token sequence
Require: *tokenizer*: Tokenizer for decoding token sequences
Require: *prefix_length*: Index marking end of user's prompt
Require: *state*: Optional persistent constraint state that includes *unit tests*.

```
1: import codeop
2: def python_syntax(...):
3:   code ← tokenizer.decode(token_ids)
4:   code ← code[prefix_length:]
5:   code ← trim_last_line(code)
6:   valid ← codeop.compile(code)
7:   score ← 1 if valid else 0
8:   return ConstraintOutput(score=score)
```

(a) Python SYNTAX

```
1: def python_comments(...):
2:   code ← tokenizer.decode(token_ids)
3:   code ← code[prefix_length:]
4:   valid ← not(("#" ∈ code) or ("'" ∈ code))
5:   score ← 1 if valid else 0
6:   return ConstraintOutput(score=score)
```

(b) No-COMMENTS

```
1: def no_explanations_constraint(...):
2:   code ← tokenizer.decode(token_ids)
3:   has_def ← code.starts_with("def")
4:   lines ← code.split("\n", 1)
5:   no_upper ← ¬get_upper(lines[1])
6:   score ← 1 if has_def & no_upper else 0
7:   return ConstraintOutput(score=score)
```

(c) No-EXPLANATIONS

```
1: import eval
2: def unit_test(...):
3:   if ¬sequence_finished(token_ids)
4:     return ConstraintOutput(score=1)
5:   end if
6:   code ← tokenizer.decode(token_ids[prefix_length :
7: ])
7:   tests ← state
8:   pass ← eval(code, tests)
9:   score ← 1 if pass else 0
10:  return ConstraintOutput(score=score)
```

(d) UNIT-TESTS

Fig. 6. Constraint function examples: SYNTAX (checks syntactic validity), No-COMMENTS (suppress comments), No-EXPLANATIONS (encourage code over natural language explanations), UNIT-TESTS (validate generated code via assertions).

These constraints can act individually or jointly during decoding, guiding the search toward syntactically valid, well-structured, and semantically meaningful programs (more details on the constraints and their use are given in the experimental evaluation in Section 7). Formally, they are composed under the product-of-experts formulation introduced earlier, where the overall constraint score is the product of individual constraint outputs. This formulation naturally supports modular composition: each constraint functions as an “expert” that can independently veto invalid continuations, while their combination yields a single, unified quality signal used by the decoding function. As a result, TREECODER enables complex constraint behaviours—such as enforcing syntax, style, and functional correctness simultaneously—without altering the underlying decoding logic.

Expensive constraints still guide search. Certain constraints such as UNIT-TESTS and EXECUTES can only be applied once a sequence has been fully generated. For simple decoding strategies such as sampling, this information is completely dismissed and we are just as likely to re-produce the same erroneous solution on the next iteration. In contrast, more sophisticated strategies such as MCTS and ASAp retain this information within the search tree, reducing the likelihood of repeating previous mistakes. This means that the harder constraints make a bigger difference for more challenging problems that the model does not get right on the first try.

6 Experimental Setup

Infrastructure. All experiments were performed on AWS EC2 g6.2xlarge instances equipped with 8vCPUs, 32 GB of RAM and an L4 GPU with 24 GB of VRAM.

Datasets. We use MBPP [6] and LiveCodeBench [24] for Python, SQLSpider [49] for SQL, and translated MBPP [33] for Rust. For Python and Rust, each problem includes unit tests, which we split

into *public* tests—appended to the problem description during generation—and *private* tests, which, together with the public tests, are used to measure true accuracy. For SQLSpider, the dataset also contains a schema and a database which is used to verify the correctness of a query by executing it and comparing the result to the reference query.

Language Models. We evaluate our framework using CODELLAMA-7B [42] instruct, CODELLAMA-13B [42] instruct, MISTRAL-7B-INSTRUCT-V0.3 [25], DEEPSEEK-CODER-7B-INSTRUCT-V1.5 [14], DEEPSEEK-R1-7B [20] and Qwen2.5-Coder-1.5B [48] models. We run models whose parameter count is above 4 B in half precision (fp16) due to VRam constraints.

Framework parameters. We use the following **termination condition** ρ for all of our experiments: we finish generation when we find 1 complete node (corresponding to the number of solutions we are interested in), or have reached 10 000 nodes which is an estimate of how many we can fit into our memory, or have no more active nodes, or when one of our nodes exceeds the length limit (to prevent infinite generation). The termination condition can be adjusted by the user. Also, the user can add a soft time constraint on the maximum runtime of the algorithm (e.g. we set this to be 10 s when finding the best LM configuration in Section 7.3).

$$\rho: \quad \#\{n \mid n \text{ is terminal} \ \& \ n\text{'s score} \geq 0\} \geq 1 \quad \text{or} \quad \#n \geq 10\,000 \quad \text{or} \\ \#\{n \mid n \text{ is active}\} = 0 \quad \text{or} \quad \#\{n \mid \text{len}(n) \geq 256\} \geq 1$$

We use the following **aggregation function** for all of our experiments: we preferably choose nodes who are complete and have a score that's higher than 0, followed by complete nodes that contain syntax errors:

$$\otimes: \quad \text{top}_5(\{n \mid n \text{ is terminal}\} \cup \{n \mid n \text{ is active} \ \& \ \text{not terminal}\} \cup \{n \mid n \text{ is not active} \ \& \ \text{not terminal}\})$$

Metrics. We report the accuracy which signifies the proportion of problems where the solution passes all public and private testcases. Conversely, the unit test constraint only uses the *public* tests to guide generation.

For later comparisons with other frameworks, we expand on this metric by using the pass@k metric, which checks whether any of the k generated solutions satisfy all unit tests. We report the total number of node expansions as a measure of how efficient our implementation is. In the context of comparing algorithms, we use the number of node expansions as an upper bound to performance. This score directly measures how many times an LLM call was made multiplied by the batchsize to produce the probability distribution over the next token.

7 Experimental Evaluation

7.1 Using TREECODER for Constraint Enforcement during Decoding

Most current interactions with LLMs for programming rely on prompt engineering to steer model behaviour and describe desired properties such as correctness or style. However, these textual constraints are interpreted heuristically and are often ignored during generation. Conversely, TREECODER enables easy experimentation with constraints directly during decoding.

To illustrate this, we evaluate CODELLAMA-7B under a range of constraint configurations (described in Section 5), all capturing information already described in the prompt. As rows 1-7 in Table 2 show, each constraint type progressively increases the precision of the generated code. In particular, unit tests yield the largest gains, while syntactic constraints alone provide marginal benefits. Lightweight constraints such as comment suppression reduce node expansions by 6.5% compared to no constraints, without degrading accuracy.

For comparison, we analyse which constraints are most often violated when specified only in the prompt, especially those related to execution and types. We find that UNIT-TESTS is most often violated (55.6%), followed by EXECUTES (39.6%) and TYPES-AND-SYNTAX (25.9%). Others are

Model	Width	Decoding	Constraints	Accuracy (%)	Expansions	Mean time taken (s)
CODELLAMA-7B	5	Sampling	NO-CONSTRAINTS	32.3	274	2.31
CODELLAMA-7B	5	Sampling	NO-COMMENTS	32.8	256	2.22
CODELLAMA-7B	5	Sampling	SYNTAX	34.0	270	2.36
CODELLAMA-7B	5	Sampling	CORRECT-SIGNATURE	36.1	254	3.57
CODELLAMA-7B	5	Sampling	TYPES-AND-SYNTAX	47.5	610	27.06
CODELLAMA-7B	5	Sampling	EXECUTES	47.1	359	5.07
CODELLAMA-7B	5	Sampling	UNIT-TESTS	60.0	417	3.90
CODELLAMA-7B	5	Sampling	All	60.7	316	3.23
CODELLAMA-7B	5	SMC	All	61.4	332	5.47
CODELLAMA-7B	5	Beam Search	All	63.0	706	16.10
CODELLAMA-7B	5	MCTS	All	65.3	1099	78.39
CODELLAMA-7B	5	ASAP	All	68.9	1158	84.05

Table 2. Performance under different decoding strategies and constraint combinations on MBPP. “All” combines SYNTAX, UNIT-TESTS and NO-COMMENTS constraints. For TYPES-AND-SYNTAX checking we use the playwright library and instruct the LM to generate type annotations (which slightly increases the length of generated program, which in turn increases the node expansions).

less frequent: NO-EXPLANATIONS (12.9%), NO-COMMENTS (4.1%), SYNTAX (2.4%), and CORRECT-SIGNATURE (1.9%). Overall, this shows that many natural-language constraints are ignored by the LM, highlighting the need for decoding-time enforcement over prompt-level guidance.

Mis-aligned models. The previous experiments evaluated constraint-guided coding on well-aligned models that generally followed code-only instructions. However, many open-source models such as MISTRAL-7B and DEEPSEEK-7B exhibit persistent misalignment: they tend to ignore prompts and interleave natural-language commentary with code, producing *explanation-code-explanation* patterns *even when explicitly asked to output only code*. This behaviour illustrates the limitations of prompt-level control and motivates decoding-time constraint enforcement.

To address this, we employ the lightweight NO-EXPLANATIONS constraint (Section 5), which enforces that outputs start with `def` and contain no capital letters after the first line. This nudges the model toward code-like continuations from the outset. Because the initial `def` token often has low model confidence, we include it in the always-considered token list and slightly increase the expansion parameter ($j=25$ rather than $j=5$) to preserve plausible continuations.

As shown in Table 3, these minimal decoding-time interventions substantially improve code validity. For both MISTRAL-7B and DEEPSEEK-7B, enforcing NO-EXPLANATIONS improves performance from near-zero compliance with basic output instructions (denoted as - in the table) to reliable code generation in over 30% and 40% of cases, respectively. For MISTRAL-7B, simple syntactic checks are insufficient. They typically trigger only after invalid prefixes denoting explanations have already been generated, making recovery difficult. This experiment reinforces the idea that even lightweight constraints, applied during decoding rather than expressed in the prompt, can realign misbehaving models toward syntactically valid and executable code.

Reasoning models. The No-Explanation and Syntax constraints are also useful for reasoning models such as DEEPSEEK-R1-7B. Although given formatting instructions in the prompt, DEEPSEEK-R1-7B does not obey them. Thus, we use these constraints to enforce runnable code, applying them only after “thinking” has finished. As shown in Table 3, performance improves from near-zero compliance (denoted as -) to reliable code generation in over 50% of cases. Moreover, the constraints nudge the LM to find solutions faster.

Model	Decoding Function	Constraints	Accuracy (%)	Expansions	Mean time taken (s)
MISTRAL-7B	Beam Search	NO-CONSTRAINTS	-	205	15.83
MISTRAL-7B	Beam Search	SYNTAX	-	603	27.68
MISTRAL-7B	Beam Search	NO-EXPLANATIONS	33.7	252	31.26
MISTRAL-7B	Beam Search	SYNTAX +NO-EXPLANATIONS	33.2	607	40.99
DEEPSEEK-7B	Beam Search	NO-CONSTRAINTS	-	190	10.50
DEEPSEEK-7B	Beam Search	SYNTAX	42.0	240	16.00
DEEPSEEK-7B	Beam Search	NO-EXPLANATIONS	40.0	325	21.00
DEEPSEEK-7B	Beam Search	SYNTAX +NO-EXPLANATIONS	42.0	320	25.04
DEEPSEEK-R1-7B	Beam Search	NO-CONSTRAINTS	-	1288	92.28
DEEPSEEK-R1-7B	Beam Search	SYNTAX	-	1151	84.32
DEEPSEEK-R1-7B	Beam Search	NO-EXPLANATIONS	54.6	1138	87.74
DEEPSEEK-R1-7B	Beam Search	SYNTAX +NO-EXPLANATIONS	52.6	1121	74.48

Table 3. Population size was set to 5, and the expansion approximation parameter j was set to 25 (for MISTRAL-7B and DEEPSEEK-7B). DEEPSEEK-R1-7B results use population size 2, $j=2$ to keep the runtime low. "-" denotes near-zero compliance with basic output instructions.

7.2 Using TREECODER to Compare Decoding Strategies

Beyond constraints, TREECODER enables users to vary the decoding strategy, revealing how different search behaviours trade off accuracy and efficiency. Among the methods tested, ASAp achieves the highest accuracy, closely followed by MCTS (rows 8–12 in Table 2). Both rely on rollouts and backtracking to refine partial hypotheses, improving solution quality but increasing cost, as reflected in the higher number of expansions. In contrast, beam search expands more nodes than sampling or SMC, as it maintains distinct beams rather than merging similar candidates. These results highlight how TREECODER facilitates systematic exploration of decoding dynamics, allowing practitioners to switch strategies within a unified framework.

7.3 Using TREECODER for Systematic Exploration of the Design Space

Beyond enabling users to manually select constraints and decoding strategies, TREECODER also supports *automatic exploration* of this design space. In particular, it integrates the optimisation algorithm described in Section 3.1 to systematically search for configurations that maximise task performance.

Finding the best configuration for CODELLAMA-7B. To illustrate this capability, we investigate the following question: *What is the the optimal decoding function, constraints, and population size that optimises the accuracy given a soft time limit of 10 seconds per sample?* We conduct this analysis for two models—CODELLAMA-7B and CodeLlama-13B—with results reported in Table 4. We address this question through a two-stage experiment that automatically identifies effective decoding configurations and then evaluates their generalisation to unseen test cases (note that we split the MBPP dataset into a smaller *validation* dataset used to search for optimal constraint combinations and *test* dataset).

Stage 1 - Searching for the optimal configuration. We employ the Optuna library [3], which implements Gaussian-process-based Bayesian optimisation, to explore five decoding functions (beam search, sampling, MCTS, ASAp, and SMC), population sizes from 1 to 5, and combinations of constraints including Python SYNTAX, UNIT-TESTS and NO-COMMENTS. In total, the search space comprises 200 unique configurations, and the optimisation uses log Expected Improvement as the acquisition function to balance exploration and exploitation. Each configuration is evaluated on the 50 validation problems of MBPP, with a soft time limit of 10 seconds per query checked after each

decoding step. Table 4 reports the best configuration found for each decoding function showing the accuracy and sample time on the test problems.

For CODELLAMA-7B, the search identifies SMC and beam search with the constraints shown in Table 4 as the most effective strategies, achieving an accuracy of 66 %, while staying within the runtime budget. For CodeLlama-13B, the strategies with the highest accuracy are sampling and beam search which achieve an accuracy of 70 %. Across decoding methods, unit tests emerge as the most influential constraint, and larger population sizes consistently improve accuracy, highlighting the benefits of maintaining multiple concurrent hypotheses.

Additionally, for CODELLAMA-7B, Figure 7 shows the Pareto frontier of accuracy against runtime on the validation dataset, allowing practitioners to select the configuration that best fits their time budget. Constrained sampling and beam search consistently dominate the frontier, offering the best accuracy-efficiency trade-off.

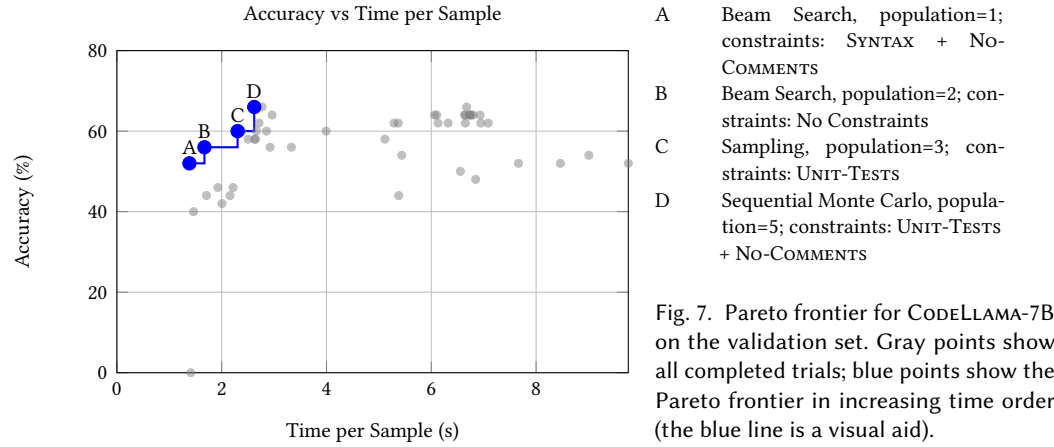
Model	Decoding Function	Constraint Config.	Pop. Size	Validation Acc.(%)	Validation Time(s)	Test Acc.(%)	Test Time(s)
CODELLAMA-7B	Sequential Monte Carlo	UNIT-TESTS +No-COMMENTS	5	66.0	2.6	59.2	3.2
CODELLAMA-7B	Beam Search	UNIT-TESTS +SYNTAX +No-COMMENTS	5	66.0	6.7	57.8	7.5
CODELLAMA-7B	Sampling	UNIT-TESTS +SYNTAX	4	62.0	2.7	55.7	3.3
CODELLAMA-7B	ASAp	No Constraints	4	54.0	5.4	49.9	6.3
CODELLAMA-7B	Monte Carlo Tree Search	UNIT-TESTS +SYNTAX	5	54.0	9.0	50.4	9.2
CODELLAMA-7B	Baseline (Sampling)	No Constraints	1	-	-	49.6	1.9
CodeLlama-13B	Sampling	UNIT-TESTS +No-COMMENTS	5	70.0	4.6	65.0	5.7
CodeLlama-13B	Beam Search	UNIT-TESTS	4	70.0	9.9	61.3	10.9
CodeLlama-13B	Sequential Monte Carlo	UNIT-TESTS	5	68.0	4.9	67.1	5.7
CodeLlama-13B	ASAp	UNIT-TESTS	4	46.0	19.8	47.5	19.8
CodeLlama-13B	Monte Carlo Tree Search	No Constraints	5	18.0	18.9	25.5	18.6
CodeLlama-13B	Baseline (Sampling)	No Constraints	1	-	-	54.1	2.4

Table 4. Performance of each decoding function on validation (tuning) and test splits of the MBPP dataset. For MCTS and ASAp, population size corresponds to the expansion parameter j .

Stage 2 – Evaluate on the Test Set. We then test the best configurations on the test dataset split of the MBPP to assess whether the optimised settings generalise to unseen programs. Table 4 also presents the corresponding results on the validation split. While overall accuracies decrease slightly, reflecting the higher difficulty of the test set, the same qualitative trends persist: *configurations that incorporate unit-test constraints and use larger populations consistently outperform the unconstrained baseline*, i.e. the default implementations for CODELLAMA-7B and CodeLlama 13B which use sampling and generate a single sequence without any constraints.

Finding the best LM configuration. Next, we run an optimisation experiment to choose the optimal LM configuration among CODELLAMA-7B, DEEPSEEK-7B and MISTRAL-7B for accuracy and a soft time limit of 10 seconds, using the test/validation split of MBPP. This experiment also includes the NO-EXPLANATIONS constraint, which suppresses the generation of natural-language explanations.

Results are reported in Table 5, where the best emerging configuration is for CODELLAMA-7B. We also show the best configuration for each model. In summary, on MBPP, the best CODELLAMA-7B configuration outperforms DEEPSEEK-7B by 14% and MISTRAL-7B-INSTRUCT-v0.3 by 24% indicating the importance of choosing the right LM for a given task. Notably, the best configuration is not the same for all models, for instance, CODELLAMA-7B does not require the NO-EXPLANATIONS constraint. In comparison to Table 4, the best decoding strategy for CODELLAMA-7B remains the



Language Model	Decoding Function	Constraint Config.	Pop. Size	Validation Acc.(%)	Validation Time(s)	Test Acc.(%)	Test Time(s)
CODELLAMA-7B	Sequential Monte Carlo	SYNTAX UNIT-TESTS	5	66.00	2.67	61.27	3.29
DEEPSEEK-7B	Beam Search	No-EXPLANATIONS UNIT-TESTS	5	52.00	9.47	50.66	9.24
MISTRAL-7B	Beam Search	No-EXPLANATIONS UNIT-TESTS	5	38.00	9.87	32.89	9.91
Baseline CODELLAMA-7B	Sampling	No Constraints	1	-	-	36.1	1.72

Table 5. Performance of each LM on validation (tuning) and test splits of the MBPP dataset.

same; however, the constraint configuration slightly differs because of the larger space and the inherent non-determinism of sampling based algorithms.

7.4 Comparison of TREECODER to Existing Frameworks

Comparison to Hugging Face. In Table 6, we compare TREECODER against the Hugging Face transformers library on the MBPP benchmark using identical decoding settings. Despite its general-purpose abstractions, TREECODER achieves virtually identical accuracy—matching pass@5 and pass@1 scores across both beam search and sampling—while maintaining near-equivalent runtime. In particular, TREECODER’s beam search is only about 25% slower (2.8 s vs. 2.1 s) and its sampling slightly faster. These results show that the framework’s modularity and flexibility introduce negligible overhead compared to specialised implementations, while supporting far richer combinations of decoding strategies and constraints.

Name	Model	Time (s)	Pass@5	Pass@1
Beam Search (TREECODER)	CODELLAMA-7B	2.79	61.0	51.0
Beam Search (HF)	CODELLAMA-7B	2.10	61.0	51.7
Sampling (TREECODER)	CODELLAMA-7B	2.73	62.0	36.7
Sampling (HF)	CODELLAMA-7B	2.76	62.0	38.4

Table 6. Runtime and accuracy comparison between TREECODER and Hugging Face Transformers on MBPP without constraints.

Comparison to IterGen. We compare TREECODER with IterGen, which supports grammar-based backtracking but explores only a single decoding trajectory at a time. We re-implement IterGen’s SQL decoding algorithm within TREECODER and reproduce its behaviour on Spider with 87.6% identical queries and nearly identical accuracy (Table 7). While TREECODER can instantiate IterGen’s decoding algorithm, the reverse is not possible as IterGen supports only one single-trajectory decoding strategy that is tightly coupled with its constraints. In particular, IterGen cannot support the comparisons across decoding strategies in Sections 7.1 and 7.2, nor the systematic exploration of decoding strategies and constraints in Section 7.3.

Name	Model	Constraints	Accuracy	Identical Queries (%)
IterGen	Qwen2.5-Coder-1.5B	SYNTAX + ALIAS	47.4	93.2
IterGen (TREECODER)	Qwen2.5-Coder-1.5B	SYNTAX + ALIAS	47.9	93.2
IterGen	Llama3-8B	SYNTAX + ALIAS	36.4	90.4
IterGen (TREECODER)	Llama3-8B	SYNTAX + ALIAS	36.2	90.4
IterGen	Llama3.2-3B	SYNTAX + ALIAS	23.3	79.1
IterGen (TREECODER)	Llama3.2-3B	SYNTAX + ALIAS	23.7	79.1

Table 7. Comparison between IterGen’s SQL decoding algorithm re-implemented in TREECODER and the original. The SYNTAX constraint enforces valid SQL syntax, while ALIAS ensures aliases refer to valid tables.

Comparison to iterative, post-hoc techniques. An alternative to constrained decoding is iterative post-hoc validation, where the LM is repeatedly invoked until an output satisfies the constraints. Unlike TREECODER, which integrates constraint checking during decoding to guide and prune search, post-hoc methods evaluate constraints only after full generation. We consider two approaches: (i) *generate-until-satisfying*, which samples until a valid solution is found, and (ii) *prompt-refinement*, which augments the prompt with failed attempts. Both use Hugging Face sampling for CodeLlama-7B; results are in Table 8.

Unlike sequential sampling, TREECODER uses *batched sampling* to generate candidates in parallel, achieving higher accuracy and lower runtime. Despite more node expansions, it is faster due to incremental SYNTAX and No-COMMENTS checking, which prunes invalid sequences early, and GPU parallelism. Prompt-refinement performs worse, as the model often makes incremental edits rather than fully regenerating programs.

Model	Max Retries/ Search Width	Decoding	Constraints	Accuracy (%)	Expansions	Mean time taken (s)
CODELLAMA-7B	5	(HF) Sampling & prompt-refinement	All	47.5	276	11.0
CODELLAMA-7B	5	(HF) Sampling & generate-until-satisfying	All	57.6	162	5.7
CODELLAMA-7B	5	(TreeCoder) Batched sampling	All	60.7	316	3.23

Table 8. Comparison between TREECODER and iterative, post-hoc techniques. “All” combines SYNTAX, UNIT-TESTS and No-COMMENTS constraints.

7.5 Generalising to More Complex Tasks

7.5.1 LiveCodeBench [24]. LiveCodeBench is generally harder than MBPP: whereas MBPP was designed as a set of short programming tasks solvable by entry-level programmers, LiveCodeBench is constructed from contest-style problems collected from platforms such as LeetCode, AtCoder,

and Codeforces, yielding a substantially more demanding evaluation setting. As shown in Table 9, adding the unit-test constraint (with a maximum runtime constraint of 30 s) improves accuracy on LiveCodeBench by 41% (from 22.8% to 32.2%). This shows that even on this more challenging benchmark, decoding-time semantic constraints can meaningfully guide the search toward more correct programs, albeit at the cost of roughly doubling the mean runtime.

Model	Width	Decoding	Constraints	Accuracy (%)	Expansions	Mean time taken (s)
Qwen2.5-Coder-7B-Instruct	5	Beam Search	-	22.8	120	11.4
Qwen2.5-Coder-7B-Instruct	5	Beam Search	Unit Tests	32.2	192	22.4

Table 9. Qwen2.5 Coder evaluation on LiveCodeBench. Maximum time per problem was set to 30 seconds.

7.5.2 Rust code generation. To evaluate whether TREECODER extends beyond Python and SQL, we conduct an experiment on Rust code generation using a translated version of the MBPP dataset [33]. Rust provides a useful stress test for constraint-based decoding because its typing and ownership rules make partial program validation more challenging than in dynamically typed languages.

Constraint based on the Rust analyzer. To enforce syntactic correctness during decoding, we integrate the rust-analyzer language server as an external constraint, which provides feedback on syntax and type errors. However, it does not provide detailed diagnostics for incomplete functions, which frequently arise during token-by-token generation. To address this, we extract the partial parse tree from rust-analyzer and automatically close open brackets before checking the program. Since this produces functions with incomplete return expressions, we append a `todo!()` statement at the end to ensure the return type remains valid.

Decoding strategy. For Rust we use a Best-First Search decoding strategy. When a completion violates the Rust constraint, the search backtracks to the last syntactically valid line and continues from that point. Since rust-analyzer takes roughly 1 s per check, we apply *lazy constraint checking*, invoking it only on the highest-scoring node in the search frontier instead of all candidates. As shown in Table 10, the Rust-analyzer constraint substantially increases the proportion of compilable programs, raising the compilation rate by 38% (63.8% to 88.3%) showing that TREECODER can incorporate language-specific tooling and constraints beyond grammar-based settings.

Model	Width	Decoding	Constraints	Compilation Rate (%)	Expansions	Mean time taken (s)
CODELLAMA-7B	10	Best-First-Search	-	63.8	103	4.7
CODELLAMA-7B	10	Best-First-Search	Rust-Analyzer	88.3	155	21.4

Table 10. Results on the translated Rust MBPP dataset.

8 Related Work

Decoding determines how an LLM explores its output space when generating text or code. While many studies introduce constrained decoding methods incorporating lexical, syntactic, or semantic rules [7, 22, 31, 37, 44], most build on a small set of core search paradigms: sampling [11, 26, 50], beam search [21–23, 31, 32, 44], SMC [10, 15, 28], and MCTS [19, 30, 40]. Various works extend these algorithms by introducing constraints during generation. Such constraints are especially valuable in code generation, where violations lead to compilation or execution errors, and can significantly improve performance when integrated into search [38]. Constraint-aware beam

search [7, 22, 31] integrates lexical, logical, or grammatical restrictions into beam expansion, while systems such as Synchronesh [37] adapt sampling to enforce semantic or structural validity. Despite their diversity, these approaches instantiate specific combinations of search policy and constraint enforcement—precisely the coupling our framework aims to decouple and generalise. Next, we focus on grammar-constrained and grammar-aligned decoding, two major paradigms for ensuring structural and semantic validity during generation.

Grammar-Constrained Decoding. Grammar-constrained decoding (GCD) aims to generate sentences that conform to a formal grammar, making it applicable to tasks such as entity disambiguation, closed information extraction, constituency parsing, conflict resolution, and code generation [17, 43]. GCD methods typically use a parser to mask tokens that would lead to invalid completions [17, 36, 43], thereby ensuring syntactic well-formedness of the generated text.

Grammar-Aligned Decoding. While GCD enforces syntactic validity, it does not ensure semantic alignment with the underlying grammar. It may assign high probability to prefixes whose valid continuations are semantically implausible, distorting the output distribution [34]. To address this, grammar-aligned decoding (GAD) methods refine decoding using grammar feedback. ASAp [34] sequentially samples under grammar constraints and adjusts token probabilities; SMC [28] maintains a population of constrained hypotheses that are periodically resampled; and MCMC [18] iteratively refines full completions via prefix-based edits. All three approaches converge to the true constrained distribution but differ in their trade-offs: MCMC can converge quickly on structured tasks, SMC benefits from GPU-friendly parallelisation at the cost of higher memory use, and ASAp and MCMC provide adaptive runtime budgets through sequential refinement.

Constrained Decoding Frameworks. The most closely related frameworks are *SynCode* [46] and *IterGen* [45]. *SynCode* constrains LLM outputs using grammar rules, while *IterGen* extends it with semantic constraints. However, *IterGen* has key limitations: (i) it supports only a single decoding strategy tightly coupled with constraints, and (ii) it is inherently single-trajectory, lacking support for batched multi-sequence generation. As a result, it cannot implement population-based algorithms (e.g. beam search, SMC, sampling). *TREECODER* addresses these limitations through a unifying abstraction that decouples decoding and constraints, enabling both trajectory- and population-based generation, modular composition, and systematic optimisation.

9 Conclusions

We introduced *TREECODER*, a modular framework for LLM constraint decoding that unifies diverse decoding strategies and constraint types within a single tree-based abstraction. Its flexible design enables automatic optimisation over decoding strategies, constraints and hyperparameters, allowing users to systematically explore and compare configurations through a shared interface. Experiments for Python, SQL and Rust show that enforcing constraints during decoding and jointly optimising decoding configurations with *TREECODER* significantly improves code-generation accuracy across models such as CodeLlama, Mistral, DeepSeek and Qwen.

Data Availability Statement

The source code for *TREECODER*, along with the benchmark datasets and scripts necessary to reproduce the evaluation in Section 7, are archived on Zenodo [39]. The source code is released under the Creative Commons Attribution 4.0 International license.

Acknowledgments

Cristina David was supported by the Royal Society University Research Fellowship URF\R\221031. Arindam Sharma was supported by the Royal Society RF\ERE\221081 and the Research Institute on Verified Trustworthy Software Systems (VeTSS) through the VeTSS Research Awards (2025).

References

- [1] Kareem Ahmed, Kai-Wei Chang, and Guy Van den Broeck. 2024. Controllable Generation via Locally Constrained Resampling. *arXiv preprint arXiv:2410.13111* (2024).
- [2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2623–2631.
- [3] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. arXiv:1907.10902 [cs.LG] <https://arxiv.org/abs/1907.10902>
- [4] Zhamri Che Ani, Zauridah Abdul Hamid, and Nur Nazifa Zhamri. 2024. The Recent Trends of Research on GitHub Copilot: A Systematic Review. In *Computing and Informatics*, Nur Haryani Zakaria, Nur Suhaili Mansor, Husniza Husni, and Fathey Mohammed (Eds.). Springer Nature Singapore, Singapore, 355–366.
- [5] Zhamri Che Ani, Zauridah Abdul Hamid, and Nur Nazifa Zhamri. 2024. The Recent Trends of Research on GitHub Copilot: A Systematic Review. In *Computing and Informatics*, Nur Haryani Zakaria, Nur Suhaili Mansor, Husniza Husni, and Fathey Mohammed (Eds.). Springer Nature Singapore, Singapore, 355–366.
- [6] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 <https://arxiv.org/abs/2108.07732>
- [7] Debangshu Banerjee, Tarun Suresh, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. CRANE: Reasoning with constrained LLM generation. *arXiv preprint arXiv:2502.09061* (2025).
- [8] Yehoshua Bar-Hillel, Micha Perles, and Eli Shamir. 1961. On formal properties of simple phrase structure grammars. *Sprachtypologie und Universalienforschung* 14 (1961), 143–172.
- [9] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810
- [10] Olivier Cappe, Simon J. Godsill, and Eric Moulines. 2007. An Overview of Existing Methods and Recent Advances in Sequential Monte Carlo. *Proc. IEEE* 95, 5 (2007), 899–924. doi:10.1109/JPROC.2007.893250
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [12] Xiang Chen, Jiacheng Xue, Xiaofei Xie, Caokai Liang, and Xiaolin Ju. 2025. A Systematic Literature Review on Neural Code Translation. arXiv:2505.07425 [cs.SE] <https://arxiv.org/abs/2505.07425>
- [13] Microsoft Corporation. 2025. Pyright: Static Type Checker for Python. <https://github.com/microsoft/pyright>. Accessed: 2025-11-13.
- [14] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang

- Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [15] Arnaud Doucet, Nando de Freitas, and Neil Gordon. 2001. *An Introduction to Sequential Monte Carlo Methods*. Springer New York, New York, NY, 3–14. doi:10.1007/978-1-4757-3437-9_1
- [16] Saibo Geng, Berkay Döner, Chris Wendler, Martin Josifoski, and Robert West. 2024. Sketch-guided constrained decoding for boosting blackbox large language models without logit access. *arXiv preprint arXiv:2401.09967* (2024).
- [17] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2024. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. arXiv:2305.13971 [cs.CL] <https://arxiv.org/abs/2305.13971>
- [18] Emmanuel Anaya Gonzalez, Sairam Vaidya, Kanghee Park, Ruyi Ji, Taylor Berg-Kirkpatrick, and Loris D’Antoni. 2025. Constrained Sampling for Language Models Should Be Easy: An MCMC Perspective. arXiv:2506.05754 [cs.AI] <https://arxiv.org/abs/2506.05754>
- [19] Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. 2025. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. arXiv:2501.04519 [cs.CL] <https://arxiv.org/abs/2501.04519>
- [20] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (Sept. 2025), 633–638. doi:10.1038/s41586-025-09422-z
- [21] Jianfei He, Shichao Sun, Xiaohua Jia, and Wenjie Li. 2023. Empirical Analysis of Beam Search Curse and Search Errors with Model Errors in Neural Machine Translation. In *Proceedings of the 24th Annual Conference of the European Association for Machine Translation*, Mary Nurminen, Judith Brenner, Maarit Koponen, Sirkku Latomaa, Mikhail Mikhailov, Frederike Schierl, Tharindu Ranasinghe, Eva Vanmassenhove, Sergi Alvarez Vidal, Nora Aranberri, Mara Nunziatini, Carla Parra Escartín, Mikel Forcada, Maja Popovic, Carolina Scarton, and Helena Moniz (Eds.). European Association for Machine Translation, Tampere, Finland, 91–101. <https://aclanthology.org/2023.eamt-1.10/>
- [22] Corentin Hokamp and Qun Liu. 2017. Lexically Constrained Decoding for Neural Machine Translation via Grid Beam Search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 1535–1546.
- [23] Liang Huang, Kai Zhao, and Mingbo Ma. 2018. When to Finish? Optimal Beam Search for Neural Text Generation (modulo beam size). arXiv:1809.00069 [cs.CL] <https://arxiv.org/abs/1809.00069>
- [24] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=chfJYC3iL>
- [25] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux,

- Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
- [26] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. arXiv:<https://www.science.org/doi/pdf/10.1126/science.abq1158> doi:10.1126/science.abq1158
- [27] Yanyu Liu, Jingying Fu, Sixiang Liu, Yitian Zou, Shouhua Zhang, and Jiehan Zhou. 2025. KV cache compression for inference efficiency in LLMs: A review. In *Proceedings of the 4th International Conference on Artificial Intelligence and Intelligent Information Processing*. 207–212.
- [28] João Loula, Benjamin LeBrun, Li Du, Ben Lipkin, Clemente Pasti, Gabriel Grand, Tianyu Liu, Yahya Emara, Marjorie Freedman, Jason Eisner, Ryan Cotterell, Vikash Mansinghka, Alexander K. Lew, Tim Vieira, and Timothy J. O'Donnell. 2025. Syntactic and Semantic Control of Large Language Models via Sequential Monte Carlo. arXiv:2504.13139 [cs.CL] <https://arxiv.org/abs/2504.13139>
- [29] João Loula, Benjamin LeBrun, Li Du, Ben Lipkin, Clemente Pasti, Gabriel Grand, Tianyu Liu, Yahya Emara, Marjorie Freedman, Jason Eisner, Ryan Cotterell, Vikash Mansinghka, Alexander K. Lew, Tim Vieira, and Timothy J. O'Donnell. 2025. Syntactic and Semantic Control of Large Language Models via Sequential Monte Carlo. In *International Conference on Learning Representations (ICLR)*.
- [30] Tingwei Lu, Yangning Li, Liyuan Wang, Binghui Lin, Jiwei Tang, Qingsong Lv, Wanshi Xu, Hai-Tao Zheng, Yinghui Li, Xin Su, and Zifei Shan. 2025. LSR-MCTS: Alleviating Long Range Dependency in Code Generation. arXiv:2504.07433 [cs.CL] <https://arxiv.org/abs/2504.07433>
- [31] Ximing Lu, Peter West, Rowan Zellers, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 4288–4299. doi:10.18653/v1/2021.naacl-main.339
- [32] Clara Meister, Tim Vieira, and Ryan Cotterell. 2021. If beam search is the answer, what was the question? arXiv:2010.02650 [cs.CL] <https://arxiv.org/abs/2010.02650>
- [33] Gabe Orlanski. 2023. BC-MBPP: Benchmark for Code Generation based on MBPP. <https://huggingface.co/datasets/gabeorlanski/bc-mbpp>. Hugging Face dataset.
- [34] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-Aligned Decoding. arXiv:2405.21047 [cs.AI] <https://arxiv.org/abs/2405.21047>
- [35] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2025. Grammar-aligned decoding. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 774, 22 pages.
- [36] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchronesh: Reliable code generation from pre-trained language models. arXiv:2201.11227 [cs.LG] <https://arxiv.org/abs/2201.11227>
- [37] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchronesh: Reliable Code Generation from Pre-Trained Language Models. arXiv preprint arXiv:2204.02805 (2022).
- [38] Henrijs Princis, Cristina David, and Alan Mycroft. 2025. Enhancing SQL query generation with neurosymbolic reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 19959–19968.
- [39] Henrijs Princis, Arindam Sharma, and Cristina David. 2026. *Artifact for TreeCoder: Systematic Exploration and Optimisation of Decoding and Constraints for LLM Code Generation*. doi:10.5281/zenodo.19440194
- [40] Isha Puri, Shivchander Sudalairaj, Guangxuan Xu, Kai Xu, and Akash Srivastava. 2025. Rollout Roulette: A Probabilistic Inference Approach to Inference-Time Scaling of LLMs using Particle-Based Monte Carlo Methods. arXiv:2502.01618 [cs.LG] <https://arxiv.org/abs/2502.01618>
- [41] Christopher D. Rosin. 2011. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence* 61, 3 (2011), 203–230. doi:10.1007/s10472-011-9258-6
- [42] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>

- [43] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. arXiv:2109.05093 [cs.CL] <https://arxiv.org/abs/2109.05093>
- [44] Torsten Scholak, Nathan Schücher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Autoregressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 7517–7535.
- [45] Shubham Ugare, Rohan Gumaste, Tarun Suresh, Gagandeep Singh, and Sasa Misailovic. 2025. IterGen: Iterative Semantic-aware Structured LLM Generation with Backtracking. arXiv:2410.07295 [cs.SE] <https://arxiv.org/abs/2410.07295>
- [46] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. Improving LLM Code Generation with Grammar Augmentation. *CoRR* abs/2403.01632 (2024). <https://doi.org/10.48550/arXiv.2403.01632>
- [47] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace’s Transformers: State-of-the-art Natural Language Processing. arXiv:1910.03771 [cs.CL] <https://arxiv.org/abs/1910.03771>
- [48] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [49] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2019. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. arXiv:1809.08887 [cs.CL] <https://arxiv.org/abs/1809.08887>
- [50] Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Jia Li, Zhi Jin, and Hong Mei. 2023. Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models. arXiv:2309.02772 [cs.SE] <https://arxiv.org/abs/2309.02772>

A Comparison To Itergen

IterGen allows traversing the search tree using grammatical units instead of tokens. This is done by incrementally parsing the currently generated sequence while keeping track of the grammatical role each token plays as well as the character start and end positions.

In figure 8 we see a comparison between our and original implementation of the SQL backtracking algorithm proposed within the Itergen paper. The algorithm uses a best-first style greedy search to generate a grammatically valid continuation until a table or column name (lines 4-7). After a column name is generated, the algorithm verifies that the most recent column name exists (lines 8-9) and backtracks to the beginning of the column name (line 10) if it does not exist. To avoid infinite loops, the process allows for a fixed number of times that the algorithm can backtrack. Each time the algorithm applies the *backward* function, the token scores that lead to the invalid completion are down-weighted. This makes those tokens less likely to be chosen again after backtracking.

We implemented the checks for the table and column names as part of the the main decoding loop instead of as a separate constraint functions. This is because the amount we want to backtrack varies depending on the type of constraint violation we have. For example, if a column violation is detected then we want to backtrack to the last column name. The tight coupling between constraints and using that information to decide how far to backtrack allows this algorithm to be very efficient at the cost of re-usability.

Both implementations are logically equivalent; however, they differ in style. In our implementation, the SQL grammar is a constraint given to the `expand_nodes` function. This modularity allows us to recognize some non-CFG languages using a combination of context free parsers.

In Itergen's implementation the forward function always uses a grammar to perform GCD until a grammatical unit is generated. The forward function also abstracts the next node's score calculation. This differs from our implementation where the score calculation is done within the main decoding function.

Overall, each framework has its own strengths and weaknesses. The itergen framework treats grammatical units as first-class citizens and is therefore tailored for implementing constraints that require navigating the underlying grammar. Forward, backward and view functions all operate on grammatical units and enable easy checking for grammatical constraints. Our framework places more focus on constraint and decoding function modularity which allows us to support a wider range of constraints and decoding strategies at the cost of not specialising for GCD. In our framework, the first order citizen is a node and backward and view functions are performed by inspecting the node's state.

B Comparison To Hugging Face

Table 11 shows that TREECODER achieves accuracy identical to Hugging Face's transformers library while maintaining near-equivalent runtime. Specifically, TREECODER's beam search is only 25% slower (2.79 s vs. 2.10 s) and its sampling slightly faster (2.73 s vs. 2.76 s) on the MBPP benchmark, with matching pass@5 scores (0.61–0.62). This demonstrates that the framework's general-purpose abstractions introduce minimal overhead compared to a specialised implementation, while providing far greater flexibility and extensibility across decoding strategies and constraint configurations.

C Additional Experimental Evaluation

C.1 Optimising for Top-5 Accuracy and Time

In Section 7.3, we identified optimal configurations for top-1 accuracy. We next examine how accuracy trades off against runtime efficiency when considering 5 generated solutions. Specifically,

Require: *Tree*: The current search tree

Require: *prompt*: Prompt for the current problem.

Require: LM: Language model

Require: ϕ_{SQL} : SQL grammar constraints

```

1: node  $\leftarrow$  Tree.root_node
2: schema  $\leftarrow$  parse_schema(prompt)
3: iters = 0
4: while not node.is_complete do
5:   nodes  $\leftarrow$  Expand(node,LM, $\phi_{SQL}$ )
6:   nodes  $\leftarrow$  score_probability(nodes)
7:   node  $\leftarrow$  nodes[0]
8:   cols  $\leftarrow$  view(child,'columns')
9:   if cols[-1]  $\notin$  schema and iters < max_iter then
10:     node  $\leftarrow$  backward(cols[-1])
11:     iters += 1
12:   continue
13: end if
14: tables  $\leftarrow$  view(child,'tables')
15: if tables[-1]  $\notin$  schema and iters < max_iter then
16:   node  $\leftarrow$  backward(tables[-1])
17:   iters += 1
18: continue
19: end if
20: end while
21: return Tree

```

(a) SQL Itergen decoding function (our implementation)

```

1: it.start(prompt,'SQL',LM)
2: schema  $\leftarrow$  parse_schema(prompt)
3: iters = 0
4: while not it.finished() do
5:   out  $\leftarrow$  it.forward('columns','tables')
6:
7:
8:   cols  $\leftarrow$  it.view('columns')
9:   if cols[-1]  $\notin$  schema and iters < max_iter then
10:     it.backward('columns')
11:     iters += 1
12:   continue
13: end if
14:   tables  $\leftarrow$  it.view('tables')
15:   if tables[-1]  $\notin$  schema and iters < max_iter then
16:     it.backward('tables')
17:     iters += 1
18:   continue
19: end if
20: end while
21: return out

```

(b) SQL Itergen (original implementation)

Fig. 8. Comparison between SQL itergen implementations. All identical lines are grayed out.

Name	Model	Time (s)	Pass@5	Pass@1
Beam Search (TREECODER)	CODELLAMA-7B	2.79	61.0	51.0
Beam Search (HF)	CODELLAMA-7B	2.10	61.0	51.7
Sampling (TREECODER)	CODELLAMA-7B	2.73	62.0	36.7
Sampling (HF)	CODELLAMA-7B	2.76	62.0	38.4

Table 11. Runtime and accuracy comparison between TREECODER and Hugging Face Transformers on MBPP without constraints.

we search for decoding configurations that maximise top-5 accuracy while maintaining low average inference time per sample.

Table 12. Optimising for top-5 accuracy (%) and time taken in seconds. The objective value is calculated by subtracting time taken from top-5 accuracy.

Decoding Function	Objective Value	Potential Function	Population Size
Sampling	64.26	(none)	5
Sequential Monte Carlo	63.94	Python Syntax	5
Beam Search	63.84	(none)	5
Monte Carlo Tree Search	57.26	(none)	5
ASAP	55.86	Comments	3

The results in Table 12 show that lightweight, population-based methods such as Sampling and SMC achieved the best trade-off between accuracy and runtime, whereas more exploratory

algorithms like MCTS and ASAP incur higher runtimes with diminishing returns. Interestingly, the optimiser often preferred unconstrained or lightly constrained decoding (e.g. Python syntax only), suggesting that strict constraints can slow decoding.

C.2 Adaptability of TREECODER

We now turn to TREECODER's broader adaptability, showing how the same framework can enable systematic studies of inference-time scaling. In particular, we investigate how increasing computational resources at inference affects accuracy. This relationship is described by inference-time scaling laws. In particular, we ask if *inference time scaling laws can be observed by changing the decoding function?* and secondly we ask *what the effect of adding constraints is to inference time scaling?*

Figure 9a, shows how many node expansions are needed for each algorithm to find 5 complete sequences when all constraints are used. We observe an increase in pass@1 accuracy when the number of node expansions increases. This is the expected result and shows that no single decoding function is better than another, meaning that the choice of the decoding function comes down to one's computational budget. In summary, we can clearly observe inference scaling laws by varying the computational budget allocated to scaling.

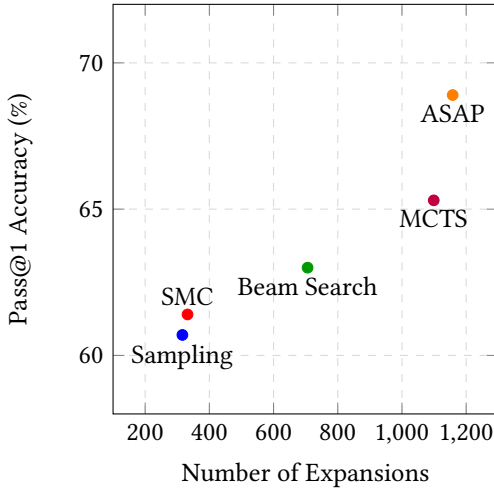
Figure 9b shows how scaling laws differ when in the presence of constraints. It shows that when no constraints are used, beamsearch quickly hits a plateau in terms of pass@1 metric. While the pass@5 metric increases as when number of node expansions is increased, this is explained by the fact that larger beam sizes get more attempts at a solution being correct (e.g. beamsize of 4 will produce 2 more solutions than beamsize of 2).

Interestingly, we see that introduction of constraints allows us to achieve much higher scaling before the accuracy levels off. This is because in highly constrained scenarios once a constraint violation has been detected in one branch, there may not be any valid prefixes in other branches that beamsearch could continue off of. This is easy to see when the beamsize is one and a constraint violation is detected, there are no other branches to switch to and thus the accuracy is equivalent.

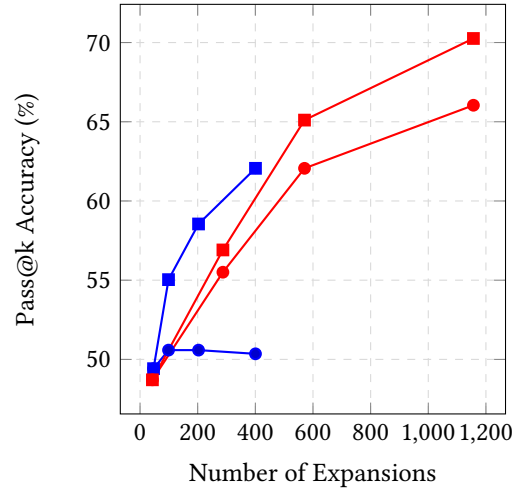
C.3 Lines of Code

Function Type	Function Name	LoC
Decoding Function	Itergen	130
Decoding Function	MCTS	113
Decoding Function	ASAP	70
Decoding Function	SMC	53
Decoding Function	Sampling	39
Decoding Function	Beam Search	39
Constraint Function	SYNTAX	26
Constraint Function	UNIT-TESTS	26
Constraint Function	NO-COMMENTS	7
Constraint Function	SQL Syntax	32
Constraint Function	NO-EXPLANATIONS	13
Constraint Function	CORRECT-SIGNATURE	49
Constraint Function	TYPES-AND-SYNTAX	49
Aggregation Function	Default Aggregation Function	58
Termination Condition	Default Termination Condition	26

Table 13. The lines of code count was obtained by excluding empty lines and comments using the cloc utility. Line length does not exceed 88 characters.



(a) Effect of changing the constraint function when all decoding functions stop after 5 complete valid nodes are found.



(b) Effect of varying beam size (1,2,4,8) on pass@1 and pass@5 accuracy. The X-axis shows the number of expansions required to find the corresponding number of complete sequences. Red lines indicate constrained decoding; blue lines indicate unconstrained decoding.

Fig. 9. Comparison of decoding performance under different scaling and constraint conditions.